Documentação de Requisitos — SAEP-PROJETO (Gestão de Estoque)

Projeto: SAEP-PROJETO · Curso técnico SENAI — Desenvolvimento de Sistemas · Repositório: github.com/Joaonascc16/SAEP-PROJETO

> Nota: esta versão foi montada a partir do que sabemos do projeto (entregas das aulas 8 e 9) e do exemplo da Aula 10. Os códigos RF-XX seguem uma numeração provisória e devem ser conferidos com a lista oficial da Aula 01.

1. Visão geral

O SAEP-PROJETO é um sistema de gestão de estoque em Java. Ele permite autenticar usuários, cadastrar produtos, registrar movimentações de entrada e saída, ordenar a listagem de produtos, alertar quando o estoque fica abaixo do mínimo e manter um histórico rastreável de cada movimentação.

2. Requisitos funcionais

2.1 Autenticação

- **RF-01 — Login válido:** o usuário informa login e senha cadastrados e é redirecionado à interface principal.
- **RF-02 — Login inválido:** com credenciais incorretas, o sistema exibe a mensagem "Login ou senha inválidos." e mantém o usuário na tela de login.

2.2 Cadastro de produtos

- **RF-03 — Cadastrar produto:** permite inserir um produto com seus campos obrigatórios (nome, estoque atual e estoque mínimo, entre outros definidos na Aula 01).
- **RF-04 — Validar campos obrigatórios:** o sistema impede o cadastro quando um campo obrigatório está vazio e informa o erro.

2.3 Gestão de estoque (aulas 8 e 9)

- **RF-05 — Ordenar produtos:** a listagem de produtos pode ser ordenada por algoritmo de ordenação, implementado como método na classe de serviço do produto (ProdutoService).

- **RF-06 — Registrar movimentação:** permite registrar entrada ou saída de um produto, com quantidade, atualizando o estoque atual.
- **RF-07 — Alerta de estoque mínimo:** quando uma movimentação deixa o estoque atual abaixo do estoque mínimo, o sistema exibe um alerta (ex.: "Estoque de Chave allen abaixo do mínimo!").
- **RF-08 — Histórico e rastreabilidade:** cada movimentação fica registrada (produto, tipo, quantidade, data/hora e usuário), permitindo consultar o histórico.

3. Regras de negócio

- **RN-01:** a saída não pode deixar o estoque atual negativo.
- **RN-02:** o alerta dispara sempre que estoque atual < estoque mínimo após a movimentação.
- **RN-03:** toda alteração de estoque gera um registro no histórico.

4. Requisitos não funcionais

- **RNF-01:** desenvolvimento em Java, com separação em camadas (classes de serviço, como ProdutoService).
- **RNF-02:** persistência em banco de dados relacional (o exemplo da Aula 10 usa o banco saep_db).
- **RNF-03:** o código deve rodar de forma estável, sem degradar após algum tempo de uso (problema relatado e a acompanhar).
- **RNF-04:** mensagens de erro e alerta claras para o usuário.

5. Requisitos aproveitáveis na Aula 10

A Aula 10 pede um caso de teste para cada área do sistema, com pelo menos um caso de sucesso e um de falha. Estes requisitos do projeto encaixam bem:

5.1 RF-07 — Alerta de estoque mínimo

É o requisito mais direto, porque a própria Aula 10 traz um exemplo de referência (CT-05). Tem resultado verificável (novo estoque e mensagem na tela) e permite um par sucesso/falha: com alerta e sem alerta.

CT-05 — Alerta de estoque mínimo disparado (RF-07)

- Pré-condição: produto "Chave allen" com estoque_atual=30 e estoque_minimo=10.

- Passos: (1) acessar Gestão de Estoque; (2) selecionar "Chave allen"; (3) escolher tipo "saída" e quantidade 25; (4) confirmar a movimentação.
- Resultado esperado: estoque_atual passa a 5 e o alerta "Estoque de Chave allen abaixo do mínimo!" é exibido.
- Resultado obtido: *preencher ao executar* · Status: *Passou / Falhou*

CT-06 — Movimentação sem alerta (RF-07)

- Pré-condição: produto "Chave allen" com estoque_atual=30 e estoque_minimo=10.
- Passos: (1) acessar Gestão de Estoque; (2) selecionar "Chave allen"; (3) escolher "saída" e quantidade 5; (4) confirmar.
- Resultado esperado: estoque_atual passa a 25 e nenhum alerta é exibido.
- Resultado obtido: *preencher ao executar* · Status: *Passou / Falhou*

5.2 RF-06 e RF-08 — Movimentação e histórico

A movimentação e o histórico mudam o estado do sistema, então dependem de uma pré-condição bem definida, exatamente o que a Aula 10 ensina. Um teste cobre os dois requisitos ao mesmo tempo.

CT-07 — Movimentação registrada no histórico (RF-06, RF-08)

- Pré-condição: produto "Chave allen" com estoque_atual=30; usuário autenticado.
- Passos: (1) acessar Gestão de Estoque; (2) registrar entrada de 10 unidades de "Chave allen"; (3) abrir o histórico de movimentações.
- Resultado esperado: estoque_atual passa a 40 e o histórico mostra o registro com produto, tipo "entrada", quantidade 10, data/hora e usuário.
- Resultado obtido: *preencher ao executar* · Status: *Passou / Falhou*

5.3 Também fáceis de cobrir

- **RF-01 / RF-02 (CT-01 e CT-02):** os exemplos prontos da própria aula; é só adaptar ao usuário e à mensagem reais do sistema.
- **RF-03 / RF-04:** cadastro válido (sucesso) e com campo obrigatório vazio (falha).
- **RF-05:** conferir que a listagem sai ordenada corretamente após aplicar a ordenação.

6. Ferramentas e ambiente de teste (item 8.1)

Preencher com os dados reais usados na execução:

- Tipo de teste: manual (ou automatizado, se for o caso)
- Navegador e versão: *a preencher*

- SGBD e versão: *a preencher*
- Versão do Java e IDE: *a preencher*
- Sistema operacional: *a preencher*

7. Rastreabilidade (requisito → caso de teste)

- RF-01 → CT-01 · RF-02 → CT-02
- RF-03 → CT-03 (a escrever) · RF-04 → CT-04 (a escrever)
- RF-06, RF-08 → CT-07
- RF-07 → CT-05 e CT-06
- RF-05 → CT-08 (a escrever)